

Naatzo

Backend del gestor de proyectos multiagente

Qué hace y cómo usarlo paso a paso (sin Docker)

Recibe una minuta o un correo desordenado y lo convierte en un proyecto listo para trabajar: tareas repartidas según la carga del equipo, un tablero Kanban propio, un repositorio con Docker y README, y un aviso al equipo. Después del arranque sigue vigilando las fechas y la carga.

Contenido

Sección	Tema
1	Qué hace el backend
2	El flujo de una ejecución: los cuatro agentes
3	Tablero propio, vigilancia de riesgos y replanificación
4	Minuta como archivo y cuentas para los miembros
5	Modos para probar sin llaves
6	Cómo usarlo sin Docker, paso a paso
7	Problemas comunes y cómo actualizar
8	Endpoints principales
9	Lo que todavía no hace

Node 20.19+ · Express · PostgreSQL · Gemini
Versión del 7 de octubre de 2026

1. Qué hace el backend

El backend es el cerebro de Naatz: recibe la minuta, coordina a cuatro agentes, guarda todo en PostgreSQL y le manda al frontend el avance en vivo. No tiene pantallas: el frontend se conecta a él por su API en **http://localhost:4000/api**.

Una regla manda en todo el proyecto:

La IA interpreta y redacta; el código calcula y ejecuta. La IA extrae requerimientos, propone tareas y escribe mensajes. Quién hace cada tarea, las fechas, los riesgos y los archivos del proyecto los decide código normal. Por eso el resultado es repetible y la IA nunca elige qué comandos se ejecutan. Todo lo que responde la IA se valida; si viene mal, se le pide otra vez mandándole el error.

En resumen, el backend se encarga de:

- Recibir la minuta (texto pegado o archivo PDF, Word o TXT) y guardarla.
- Coordinar a los cuatro agentes en orden, guardar lo que decide cada uno y pedir aprobación antes de crear cosas afuera.
- Hablar con los servicios externos: el modelo de lenguaje (Gemini), GitHub y Discord.
- Mandar al frontend el avance en vivo, para que se vea qué agente está trabajando.
- Llevar el tablero Kanban del equipo, con cuentas para cada miembro.
- Vigilar el proyecto después del arranque (retrasos y sobrecarga) y replanificar cuando algo cambia.

2. El flujo de una ejecución

Se crea un equipo (personas con sus habilidades y horas por semana) y un proyecto con la minuta. Al arrancar una ejecución, los agentes trabajan en este orden:

Paso	Qué hace la IA	Qué hace el código
Analista	Lee la minuta y extrae objetivo, tecnologías, requerimientos, fecha de entrega y dudas.	Valida la respuesta, normaliza el stack contra las plantillas (Node, NodeJS o Express pasan a node-express) y pone valores por defecto a lo que falta, anotándolo como duda.
Planificador	Divide el proyecto en módulos y tareas con horas, prioridad y dependencias.	Asigna cada tarea a quien tiene la habilidad y menos carga, calcula fechas en días hábiles, detecta sobrecarga y entregas imposibles, y propone reasignaciones.
Aprobación	-	La ejecución espera a que una persona revise el plan y lo apruebe.
DevOps	Redacta solo el texto del README.	Arma el proyecto desde plantillas probadas (Node + Express, React + Vite, docker-compose con Postgres 16), crea el repositorio en GitHub en un solo commit o deja un ZIP si GitHub falla.
Notificador	Redacta un resumen de dos o tres líneas.	Pone los links, números y fechas, y manda el aviso al equipo por Discord.

Cada paso queda guardado (qué recibió, qué decidió, qué modelo usó y cuánto tardó). Si un paso falla, se reintenta desde ese paso. Los estados son: en cola, analizando, planeando, esperando aprobación, creando entorno, notificando y completado (o fallido).

3. Tablero, vigilancia y replanificación

Tablero propio (en lugar de Trello)

- Listas configurables por proyecto: nace con Por hacer, En progreso, En revisión y Hecho, y el equipo puede crear, renombrar, reordenar y borrar listas. Cada lista tiene una etapa, para que la carga y los riesgos sigan funcionando con listas propias como "Bloqueado".
- Tarjetas que se mueven entre listas, se archivan y restauran, con etiquetas de colores, checklist y comentarios.
- Historial de actividad ("Laura movió X a En progreso") y cambios en tiempo real: dos personas con el tablero abierto ven los movimientos al instante.
- Gráfica de carga: horas asignadas contra horas disponibles por persona.

Vigilancia después del arranque

Cada 10 minutos revisa las tareas y avisa si alguna está vencida, vence en menos de 24 horas, no ha empezado a mitad del plazo, o si alguien pasó del 100% de carga. No repite el mismo aviso. Para la demo hay un reloj simulado que se puede adelantar (por ejemplo, 6 días) para ver aparecer los avisos.

Replanificar con una frase

Se le escribe algo como *"Ana no puede esta semana"*, *"Luis ahora tiene 4 horas por semana"* o *"el cliente adelantó la entrega cinco días"*. La IA lo convierte en cambios (ausencia, horas o fecha de entrega) y el código recalcula:

- Las tareas en curso se quedan con quien las trabaja.
- Las tareas por hacer que caían dentro de una ausencia pasan a otra persona con la misma habilidad; si nadie más la tiene, esperan y se recorren las fechas.
- Las fechas se saltan los días de ausencia y la capacidad se descuenta.
- La IA explica qué se movió, se registra en la actividad y se avisa al equipo.
- Con la opción de vista previa muestra el resultado sin guardar nada, para confirmarlo antes.

4. Minuta como archivo y cuentas para los miembros

Minuta como archivo

Además de pegar el texto, se puede subir la minuta como PDF, Word (.docx) o TXT, de hasta 5 MB, al crear el proyecto o después. El backend extrae el texto y revisa que el archivo sea lo que dice ser. Un PDF escaneado (una foto del texto) no se puede leer, y se pide pegar el texto.

Cuentas para los miembros

Quien crea el equipo es el dueño. Puede invitar a cada miembro con un link que vence en 7 días; la persona se registra con ese código y entra al tablero con su propio login. Tiene una vista de "mis tareas" con lo que le toca y sus fechas, y lo que hace aparece con su nombre en la actividad.

Acción	Dueño	Miembro
Ver equipo, proyectos, tablero (en vivo), actividad, ejecuciones, entorno y avisos	Sí	Sí
Crear, editar, mover, archivar y etiquetar tarjetas; checklist y comentarios	Sí	Sí
Registrar o quitar ausencias	De cualquiera	Solo las tuyas
Arrancar, aprobar o reintentar ejecuciones; replanificar	Sí	No
Crear o borrar proyectos, cambiar la minuta, borrar tarjetas	Sí	No
Administrar listas, etiquetas, miembros e invitaciones	Sí	No

Quien no pertenece al equipo no ve nada: para él, el proyecto no existe.

5. Modos para probar sin llaves

Sin...	Qué pasa
Llave de Gemini (LLM_API_KEY)	Usa respuestas de ejemplo (modo mock), pensadas para la minuta de ejemplo. La replanificación entiende frases típicas por palabras clave.
GitHub (GITHUB_TOKEN y GITHUB_OWNER)	Genera el proyecto como ZIP descargable en lugar del repositorio.
Discord (DISCORD_WEBHOOK_URL)	Los avisos se guardan en el historial para mostrarlos en pantalla.
Internet, en modo demo	Si Gemini no responde, cada agente usa la respuesta de ejemplo y lo deja registrado.

Con DEMO_MODE=true existen además tres rutas para presentar: cargar un equipo de 4 personas con una minuta de ejemplo, adelantar el reloj simulado y borrar los datos de la demo.

6. Cómo usarlo sin Docker, paso a paso

Esta misma guía viene en texto simple en **COMO_USARLO_SIN_DOCKER.txt**, para abrirla con el Bloc de notas.

Paso 1. Instala lo necesario

- Node.js 20.19 o más nuevo. Recomendado: la versión LTS de nodejs.org.
- PostgreSQL 13 o más nuevo. En Windows, el instalador de postgresql.org. Durante la instalación anota la contraseña del usuario postgres y deja el puerto 5432. Si no quieres instalarlo, puedes usar Supabase (Paso 4B).
- Opcional: Git Bash (viene con Git para Windows), para correr el script de demo.

Comprueba Node en una terminal:

```
node -v
npm -v
```

Paso 2. Descomprime el proyecto

- Descomprime naatzo-backend.zip, por ejemplo en el Escritorio. Queda la carpeta naatzo-backend.
- Abre una terminal dentro de esa carpeta. En Windows: entra a la carpeta en el Explorador, escribe powershell en la barra de dirección y presiona Enter. En Mac o Linux: abre Terminal y usa cd.

Paso 3. Instala las dependencias

```
npm install
```

Tarda uno o dos minutos y crea la carpeta node_modules.

Paso 4A. Crea la base de datos (Postgres en tu computadora)

Con pgAdmin: clic derecho en Databases > Create > Database, y ponle de nombre naatzo.

O desde PowerShell (cambia 17 por tu versión de Postgres; te pide la contraseña de postgres):

```
& "C:\Program Files\PostgreSQL\17\bin\psql.exe" -U postgres -c "CREATE DATABASE naatzo"
```

En Mac o Linux:

```
createdb -U postgres naatzo
```

Paso 4B. (Alternativa) Usa Supabase en lugar de Postgres local

- En Supabase: Project Settings > Database > Connection string > Session pooler, y copia la URL.
- En el .env (Paso 5) pon esa URL en DATABASE_URL y DATABASE_SSL=true.
- Si es el mismo proyecto de Supabase de KIBO 1, agrega DB_SCHEMA=naatzo para que las tablas queden aparte.

Paso 5. Configura el archivo .env

Copia la plantilla y ábrela con el Bloc de notas:

```
Copy-Item .env.example .env
notepad .env
```

(En cmd es `copy .env.example .env`; en Mac o Linux, `cp .env.example .env`.) Cambia como mínimo:

```
DATABASE_URL=postgres://postgres:TU_CONTRASENA@localhost:5432/naatzo
DATABASE_SSL=false
JWT_SECRET=escribe-aqui-un-texto-largo-de-al-menos-16-caracteres
DEMO_MODE=true
```

- Si tu contraseña tiene caracteres especiales, escríbelos codificados: @ es %40, # es %23, : es %3A, / es %2F y ? es %3F.
- Las llaves (LLM_API_KEY, GITHUB_TOKEN, DISCORD_WEBHOOK_URL) pueden quedar vacías: el backend usa respuestas de ejemplo y guarda los avisos para mostrarlos en pantalla.
- Al guardar, el archivo debe llamarse .env y no .env.txt. En el Bloc de notas elige "Todos los archivos" en el tipo.

Paso 6. Crea las tablas

```
npm run migrate
```

Debe decir: [migrate] 7 migracion(es) aplicada(s) en el esquema public. Si lo corres otra vez dice "La base ya estaba al día"; no borra nada.

Paso 7. Arranca el servidor

```
npm start
```

(`npm run dev` hace lo mismo y además se reinicia solo cuando editas el código.) Debe mostrar:

```
[naatzo] API escuchando en http://localhost:4000
[naatzo] LLM: mock | demo: true
[riesgos] Revision programada: */10 * * * *
```

Deja esa terminal abierta mientras lo usas. Para detenerlo: Ctrl + C.

Paso 8. Comprueba que funciona

Abre en el navegador:

- `http://localhost:4000/api/health`: debe responder `{"ok":true,...}`
- `http://localhost:4000/api/health/db`: debe decir "conectada"
- `http://localhost:4000/api/health/integrations`: qué está configurado (mock, GitHub, Discord)

La raíz `http://localhost:4000` solo dice que la API está arriba: es un backend, no tiene páginas.

Paso 9. Corre la demo completa

En otra terminal (en Windows, Git Bash), dentro de la carpeta `naatzo-backend`:

```
bash scripts/demo.sh
```

Recorre todo el flujo y lo muestra en pantalla. Crea dos usuarios de prueba:

- Dueño: `demo@naatzo.dev` / `naatzo-demo-123`
- Miembro: `laura@naatzo.dev` / `laura-demo-123`

Paso 10. Úsalo desde Postman, Thunder Client o el frontend

- Regístrate: POST `http://localhost:4000/api/auth/register` con el JSON `{"name":"Tu nombre","email":"tu@correo.com","password":"minimo8caracteres"}`. Regresa un token.
- En las demás peticiones agrega el header `Authorization: Bearer <token>`.
- Atajo: POST `/api/demo/seed` carga un equipo de 4 personas y la minuta de ejemplo.
- Después: POST `/api/projects/:id/runs` arranca a los agentes, y GET `/api/projects/:id/board` muestra el tablero.
- En PowerShell, para los ejemplos con curl usa `curl.exe` (no `curl`).

Paso 11 (opcional). Usar llaves reales

- Gemini: `LLM_API_KEY`, una llave de Google AI Studio.
- GitHub: `GITHUB_TOKEN` (fine-grained, con permiso para crear repositorios y escribir contenido), `GITHUB_OWNER` y `GITHUB_OWNER_TYPE` (org o user).
- Discord: `DISCORD_WEBHOOK_URL` (Ajustes del canal > Integraciones > Webhooks).
- Reinicia el servidor después de editar el `.env` (Ctrl + C y `npm start`), y revisa `/api/health/integrations`.
- No subas el `.env` a GitHub ni lo pegues en el chat del grupo: ahí van las llaves.

7. Problemas comunes y cómo actualizar

Si ves...	Qué hacer
"node" no se reconoce como comando	Node no está instalado, o la terminal se abrió antes de instalarlo. Ciérrala y ábrela de nuevo.
[env] Configuración inválida	Falta o está mal una variable del <code>.env</code> . El mensaje dice cuál.
ECONNREFUSED ... 5432	Postgres no está corriendo. Win + R, <code>services.msc</code> , e inicia el servicio <code>postgresql-x64-XX</code> .
password authentication failed	La contraseña en <code>DATABASE_URL</code> no es la del usuario postgres.
database "naatzo" does not exist	Falta crear la base (Paso 4A).
EADDRINUSE ... 4000	El puerto 4000 ya está ocupado (otro servidor abierto). Ciérralo o cambia PORT en el <code>.env</code> .
Errores de SSL o certificado	En Postgres local usa <code>DATABASE_SSL=false</code> ; en Supabase, <code>true</code> .
El frontend da error de CORS	Agrega su dirección a <code>CORS_ORIGINS</code> (separadas por coma) y reinicia.
ERR_REQUIRE_ESM u otro error raro al arrancar	Tu Node es más viejo que 20.19. Actualízalo.

Actualizar a una versión nueva del ZIP

1. Reemplaza los archivos con los del ZIP nuevo, pero conserva tu `.env`.
2. `npm install`
3. `npm run migrate` (aplica solo las migraciones nuevas; no borra tus datos)
4. `npm start`

8. Endpoints principales

Todas las rutas van bajo /api, responden JSON con la forma { ok, ... } y piden el header Authorization: Bearer <token>, menos /auth y /health. La lista completa está en el README.

Método y ruta	Qué hace
POST /auth/register, /auth/login	Crea la cuenta o inicia sesión; regresa el token
GET /auth/me	Usuario y sus equipos (dueño o miembro)
POST /teams, POST /teams/:id/members	Crea el equipo y agrega miembros (habilidades y horas)
POST /projects	Crea el proyecto con la minuta (texto o archivo)
POST /projects/:id/runs	Arranca a los agentes
GET /runs/:id/events?token=...	Avance en vivo (SSE)
POST /runs/:id/approve	Aprueba el plan; siguen DevOps y Notificador
GET /projects/:id/board	Tablero: listas, tarjetas, carga, riesgos y propuestas
PATCH /tasks/:id/move	Mueve una tarjeta de lista
POST /tasks/:id/comments, /checklist	Comenta o agrega elementos al checklist
POST /projects/:id/replan	Replanifica con una frase (con vista previa)
POST /members/:id/invite	Invita a un miembro (link que vence en 7 días)
GET /me/tasks	Mis tareas en todos mis equipos
GET /projects/:id/environment/zip	Descarga el proyecto generado
POST /demo/seed, /demo/clock, /demo/reset	Rutas de demo (solo con DEMO_MODE=true)

9. Lo que todavía no hace

- No tiene interfaz: el frontend es aparte (por ejemplo, la pantalla que abre el link de invitación).
- No se conecta con Jira, Telegram ni correo; los avisos salen por Discord o se muestran en pantalla.
- No lee PDF escaneados (necesitaría OCR) ni guarda archivos adjuntos en las tarjetas.
- El agente administrado (Antigravity) para verificar el repositorio queda para después del MVP.
- La interpretación con Gemini real necesita una llave; sin ella, todo corre con respuestas de ejemplo.